A New Rendering and Plume Simulation Framework for Visualizing Taal Volcano Eruption Plumes

A Thesis Proposal

presented to

the Department of Software Technology
College of Computer Studies
De La Salle University

In partial fulfillment
of the requirements for the degree of

Bachelor of Science in Computer Science
Major in Software Technology

by

BERNANDINO, Adrian Rafael
BORROMEO, Anton Miguel
GALAN, Russell Emmanuel
GANAYO, Reinman Geoffrey

Neil Patrick Del Gallego, Ph.D.
Adviser

July 28, 2026

Abstract

The Taal Volcano in the Philippines remains one of the country's most active volcanoes due to its recurring eruptive activity, highlighting the continued need for effective tools that support hazard communication, disaster preparedness, and scientific visualization. Interactive three-dimensional computer graphics simulations have demonstrated their potential to provide intuitive visual representations of volcanic eruption plumes while allowing users to explore eruption scenarios in an immersive virtual environment. AnitoPlume, an existing graphics-based Taal Volcano plume simulator, has successfully demonstrated real-time visualization capabilities using a particle-based simulation model; however, it relies on legacy graphics rendering technologies and software architectures that limit extensibility, maintainability, and the efficient utilization of modern graphics hardware. This study proposes a modern 3D graphics rendering framework for interactive Taal Volcano plume simulation by porting and extending AnitoPlume, redesigning its rendering architecture while preserving its established particle-based simulation model and core visualization techniques. The proposed framework aims to modernize the rendering pipeline, graphics resource management, shader implementation, and software architecture to provide a more scalable, maintainable, and extensible platform for future graphics-based scientific visualization applications. A design and development research methodology will be employed to reimplement AnitoPlume using a contemporary graphics rendering framework and evaluate its rendering performance, graphics resource utilization, scalability, and visual fidelity through quantitative performance metrics and qualitative visual assessment.

Keywords: Computer graphics, 3D graphics, Graphics rendering, Volcanic plume simulation, Taal Volcano, Particle systems, AnitoPlume

Chapter 1

Introduction

1.1 Background of the Study

Volcanic eruptions are among the most destructive natural hazards, producing ash plumes, pyroclastic flows, and volcanic gases that threaten nearby communities and critical infrastructure. Effective visualization of these hazards plays an important role in disaster preparedness, scientific communication, and public education. Traditionally, volcanic simulations have relied on numerical models that prioritize scientific accuracy for hazard prediction and analysis. While these models provide valuable quantitative information, they are often computationally intensive and lack the interactive capabilities necessary for real-time visualization (Rossant & Rougier, 2021).

The original AnitoPlume was developed as a real-time 3D simulator for Taal Volcano. Replicating the 2020 eruption and providing an interactive, and immersive, application for use in disaster communication and awareness (Del Gallego et al., 2026). The application can be broken down into its three major components: a terrain model, the plume simulations, and the UI. The terrain component renders a detailed elevation model of Taal Volcano, with collisions and maps, serving as the basis for plume behavior regarding terrain. The plumes use a Lagrangian, particle-based approach to represent eruption dynamics through smoke layers, free spheres, falling spheres, and stagnation states. These allow the plume to rise, spread, collapse, and interact with wind and terrain. The user interface provides control over eruption parameters, wind settings, camera and display modes, etc.

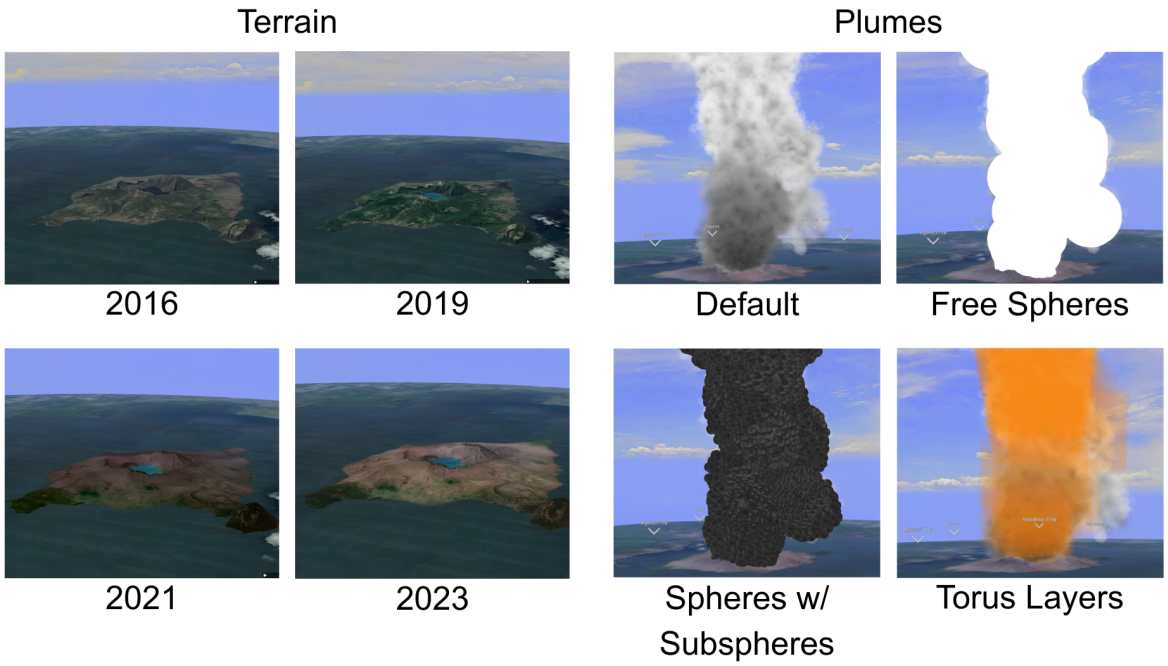

Figure 1: Terrain models of Taal Volcano inside the Anito plume and Different plume simulations of AnitoPlume. (Del Gallego et al., 2026).

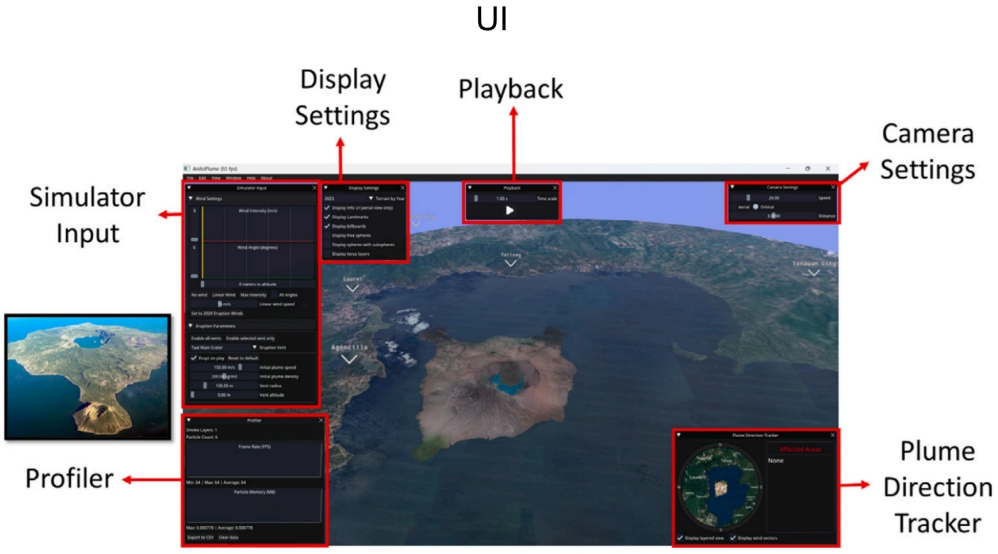

Figure 2: User Interface of AnitoPlume. The Taal volcano model is rendered in the center of the simulator. The photo reference is on the left. (Del Gallego et al., 2026).

Graphics Application Programming Interfaces (APIs) define how applications communicate rendering operations to the graphics processing unit (GPU). Unlike traditional implicit graphics APIs, modern explicit graphics APIs require applications to explicitly manage rendering state, resource allocation, and command submission, thereby reducing driver overhead and improving utilization of modern multicore processors and graphics hardware (Ping et al., 2026).

Despite the theoretical performance advantages of modern rendering pipelines, a significant challenge remains: the architectural enhancement of established scientific tools. Therefore, investigating how GPU control impacts rendering throughput, memory bandwidth utilization, and overall visual fidelity provides essential insights for the future development of highly scalable scientific visualization systems(Unterguggenberger, Kerbl, & Wimmer, 2022). While AnitoPlume demonstrates that a volcanic plume can be visualized interactively at real-time framerates, its existing graphics architecture limits the possible terrain and plume optimizations.

Figure 3: Overview of older and modern graphics APIs. From top to bottom, left to right, OpenGLES, OpenGL, DirectX 11, MetalFX, Vulkan, DirectX 12 (TechArx Gaming Staff, 2016).

The proposed AnitoPlume V2 therefore aims to port the visual system to a new rendering framework, and revisit the existing plume simulation logic. Improving on, but not limited to: GPU control and utilization, shading flexibility, rendering efficiency, and simulation logic improvements. In doing so, the project will improve how the terrain and plumes are rendered, with emphasis on higher fidelity terrain models and plume simulations at least as detailed as the current representation.

1.2 Research Objectives

This study aims to port and extend AnitoPlume to a modern graphics framework, preserving its established particle-based simulation model, evaluate the resulting

improvements, and assess whether the system maintains an interactive and usable experience.

The specific objectives of this research are as follows:

1. To benchmark GPU resource synchronization and cross-API shader translation for AnitoPlume's volumetric plume data, establishing a baseline for porting the simulator to the new rendering framework;
2. To redesign AnitoPlume's software architecture by migrating to alternative rendering frameworks that support explicit-based architectures, physically-based lighting (PBL), such as but not limited to Vulkan and DirectX 12;
3. To analyze the rendering efficiency of the modernized pipeline against the previous implementation using GPU profiling metrics, such as, but not limited to, frame pacing, shader execution time, memory bandwidth utilization, and overall render throughput, under identical hardware constraints; and
4. To validate the overall user experience of the modernized simulator by conducting usability testing with target beneficiaries.

1.3 Scope and Limitations of the Research

This study is limited to porting and extending AnitoPlume, an existing Taal Volcano plume visualization system, from its current graphics API (OpenGL) to a modern graphics API, such as Vulkan or DirectX 12. In contrast to legacy APIs, where the graphics driver automatically manages GPU state, memory allocation, and synchronization on the application's behalf, APIs require the application to directly define and control these aspects, including Pipeline State Objects (PSOs), command lists, and GPU resource synchronization. This shift is the basis for the "explicit" rendering architecture investigated in this research. The study specifically examines pipeline state management, command recording, shader portability across graphics APIs, and GPU resource synchronization, while strictly preserving AnitoPlume's existing particle-based plume simulation model. Modifications to the underlying scientific simulation algorithms, volcanic plume physics, or eruption modeling parameters are outside the scope of this work. Furthermore, the study is scoped to graphics API-level constructs rather than the abstractions of any single third-party rendering framework or engine, ensuring that the resulting architecture and findings remain applicable across explicit-API implementations rather than being dependent on one specific framework.

The usability testing phase will involve 10 to 20 voluntary participants, recruited according to a tiered priority order: PHIVOLCS personnel will be prioritized as the primary participant group given their domain expertise in volcanology; should this pool be insufficient, recruitment will expand, in order, to individuals who witnessed a Taal Volcano eruption firsthand, then to volcano/geohazard simulation enthusiasts, and lastly to computer science students or developers with a relevant background in 3D graphics or software development. Recruitment will draw on coordination with PHIVOLCS, community outreach for eruption witnesses, online volcano/geohazard simulation communities, and on-campus recruitment within DLSU computer laboratories. Data collection will take place within the DLSU College of Computer Studies (CCS) laboratory environments, at PHIVOLCS facilities or an agreed external venue for agency personnel, or in controlled remote testing environments for participants unable to be physically present. After completing a standardized set of interaction tasks with the ported simulator, participants will complete an anonymous, self-administered questionnaire using the System Usability Scale (SUS) to evaluate application responsiveness, stability, and visual parity relative to the original AnitoPlume system. This phase is expected to take 2 to 4 weeks within the systems evaluation timeframe of the study.

This research is limited to a single case study, the AnitoPlume Taal Volcano plume simulator, and its findings on rendering performance and usability are not guaranteed to generalize to other volcanic visualization systems or simulation platforms. Benchmarking results are further constrained to the specific hardware configurations used during testing and may not be representative of performance across all GPU vendors or consumer hardware. Finally, given the sample size of 10 to 20 participants drawn from a mix of domain-expert and general-user profiles, usability findings are intended to provide indicative, rather than statistically generalizable, insight into user experience.

1.4 Significance of the Research

This study contributes to both computer graphics research and scientific visualization by providing empirical evidence on the application of modern graphics architectures to real-time volumetric scientific visualization, while also offering immediate, practical value to Philippine disaster risk reduction efforts.

For Philippine communities and disaster management stakeholders, this study offers direct, immediate benefits. Taal Volcano remains under close monitoring due to its recurring eruptive activity, yet accessible, interactive 3D tools for com-

municating volcanic hazards to the public and to local decision-makers remain limited. By modernizing and extending AnitoPlume into a more performant, maintainable, and scalable system, this study lays the groundwork for AnitoPlume V2 to potentially serve as one of the first accessible 3D visualization tools for volcanic hazard communication in the Philippines. Such a tool could support disaster preparedness education, hazard awareness campaigns, and public information efforts led by local government units (LGUs) and agencies such as PHIVOLCS, by allowing communities and responders to visualize eruption scenarios in a more intuitive and immersive manner than static hazard maps alone. While this study does not directly evaluate deployment with disaster response agencies or affected communities, it establishes the technical foundation necessary to pursue such applications in future work.

For researchers in computer graphics, the study offers a detailed case study demonstrating the architectural modifications required when transitioning between two fundamentally different graphics APIs.

For developers of scientific visualization software, the findings may serve as a reference for future projects that require designing high-performance rendering systems for similarly data-intensive, real-time visualization applications.

For future researchers, this work establishes a scalable rendering foundation that supports integrating advanced graphics techniques, including hardware-accelerated ray tracing, compute shaders, GPU-driven rendering, and other contemporary visualization methods. Consequently, the study extends the long-term applicability of the original AnitoPlume project beyond its initial legacy implementation, while positioning it as a potential stepping stone toward practical, real-world disaster communication tools for volcanic hazards in the Philippines.

Chapter 2

Related Works

2.1 Taal Volcano Primer

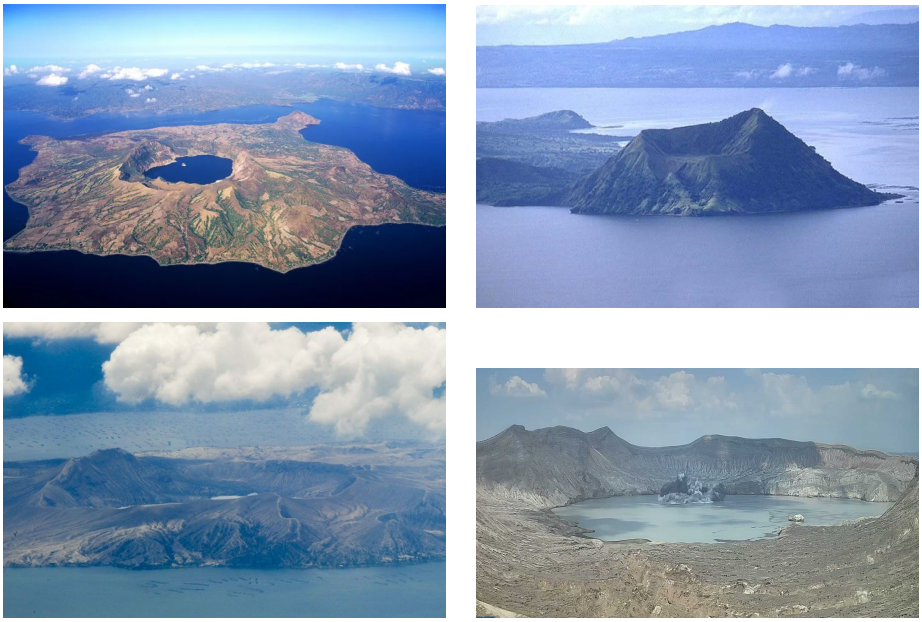

Figure 4: From left to right, top to bottom: Taal volcano during 2015, 2019, 2023, and 2026 (TechArx Gaming Staff, 2016; Flores, 2019; Daguno-Bersamina, 2023; Mallari Jr., 2026).

Taal Volcano is a complex volcano located in southwestern Luzon, Philippines, situated in the middle of Taal Lake (Smithsonian Institution, 2026). Figure 3 illustrates the volcano across different time periods, including its primary eruption

phases. As one of the most active volcanoes in the country, the Philippine Institute of Volcanology and Seismology (PHIVOLCS) has recorded 36 historical eruptions at Taal (PHIVOLCS, 2024). Its frequent activity poses a major threat to local communities and the densely populated Metro Manila area. Although official hazard maps designate only the volcano island as a Permanent Danger Zone (PDZ) and outline a high-risk perimeter of 15 km, Taal's actual danger extends much further due to the regular release of toxic gases. Consequently, Metro Manila remains highly vulnerable to these emissions despite being located approximately 81 km away from the volcanic center.

This vulnerability was clearly demonstrated during the January 12, 2020 eruption, when a phreatomagmatic event occurred at the main crater, producing a massive column of ash and steam. The severity of the eruption led PHIVOLCS to raise the alert status to Level 4, indicating that a hazardous eruption was imminent. The event was classified as a Volcanic Explosivity Index (VEI) 4 eruption, depositing an estimated tephra fallout volume of 0.097 km^3 (Balangue-Tarriela et al., 2022). Due to the immediate risks, a mass evacuation of roughly 376,000 people was carried out in the surrounding areas, including the provinces of Batangas and Cavite (Renzo, 2020; Del Castillo, Paraiso, Vicente, Jamero, & Narisma, 2020).

The 2020 event is the only major eruption of Taal to be thoroughly documented, thanks to modern communication and technologies like accessible digital cameras, crowdsourcing, and high-resolution seismic sensors. This availability of real eruption footage and geographical data provides a strong foundation for developing visual simulations of the volcano.

2.2 Geoscientific Modeling and Volcanic Crisis Communication

2.2.1 Limitations of Current Numerical Models

In volcanology, the standard approach for evaluating risk and hazard zones relies on numerical simulations that model the physics of ash transport and tephra fallout. Volcanologists widely use Eulerian models such as FALL3D (Folch, Costa, & Macedonio, 2009), Tephra2 (Bonadonna, 2006; Connor & Connor, 2006), and TephraProb (Biass, Bonadonna, Connor, & Connor, 2016), alongside Lagrangian models like HYSPLIT (Stein et al., 2015), for scientific analysis and hazard mapping. These models are highly valued for their quantitative accuracy and their

capacity to incorporate atmospheric variables.

However, these simulations are built for expert analysis rather than public situational awareness, requiring high-performance computing and substantial processing time. While these methods are essential for long-term hazard forecasting, they lack the interactive flexibility and 3D immersion needed for real-time visualization applications. To address this, our proposed software, AnitoPlume, serves as a computer graphics application that complements these numerical models by offering an interactive testbed for 3D volcanic eruption visualizations of Taal Volcano.

2.2.2 Advancements in 3D Hazard Visualization

Traditionally, volcanic hazards and crisis information are communicated using 2D maps that display contour lines and heat maps for ash distribution, pyroclastic flow boundaries, and tephra thickness (Nave, Isaia, Vilardo, & Barclay, 2010; Thompson, Lindsay, & Gaillard, 2015; Fearnley & Beaven, 2018; Esposti Ongaro, Cerminara, Charbonnier, Lube, & Valentine, 2020). Although 2D representations remain the standard for scientific experts, research indicates that integrating "spatial familiarity" is critical for prompting effective public responses (Preppernau & Jenny, 2015). Comparative studies further demonstrate that general users often prefer 3D perspectives over 2D maps, as 3D views make it easier to understand complex terrain features and identify viable evacuation routes (Thompson, Lindsay, & Leonard, 2017; Driedger et al., 2024).

To improve public communication, several research initiatives have explored immersive and interactive applications using computer graphics (Zhang, Kong, Li, Wang, & Qin, 2017; Lastic et al., 2022; Pretorius et al., 2024), virtual reality (VR) (Tibaldi et al., 2020; Asgary, Bonadonna, & Frischknecht, 2019; Li, 2023), or gamification (Kerlow, Pedreros, & Albert, 2020). Tools such as VolcanoVR (Hansen et al., 2025), VRVOLC (Delage et al., 2021), and HoloVulcano (Asgary et al., 2019) utilize commercial game engines to let stakeholders view complex volcanic phenomena in safe, immersive environments.

Distinct from these commercial engine approaches, the original AnitoPlume framework was developed using a custom, open-source stack (C++, OpenGL, Eigen, dearIMGUI) to bypass general-purpose engine abstractions and optimize real-time rendering performance. Furthermore, while generic eruption simulations exist (Lastic et al., 2022; Pretorius et al., 2024), AnitoPlume specifically targeted the high-risk Taal "Decade Volcano" by integrating LiDAR and OpenTopography terrain datasets with real-time plume dynamics. Building directly upon this

specific, site-tailored approach, our work modernizes this legacy architecture by replacing its original rendering backend with a next-generation graphics API, significantly enhancing visual fidelity and pipeline efficiency.

2.3 Geosciences and computer graphics

Volcanic modeling paradigms in geosciences and computer graphics differ significantly in both methodology and primary intent. Within geoscientific and volcanological research, the priority is predictive accuracy and scientific forecasting. In this context, numerical solvers are essential for quantifying physical parameters such as mass eruption rates and tephra dispersal (Kavanagh, Engwell, & Martin, 2018; Poland & Anderson, 2020; Esposti Ongaro et al., 2020).

Conversely, in the field of computer graphics, volcanic modeling is treated as a subfield of real-time rendering and procedural simulation, focusing primarily on particle systems and fluid dynamics (Stora, Agliati, Cani, Neyret, & Gascuel, 1999; Roth & Guritz, 1995; Lastic et al., 2022; Li, 2023; Pretorius et al., 2024). Research in this domain emphasizes real-time user interactivity and immersion, meaning that visual effects are often approximations and physical models are simplified. Consequently, these techniques prioritize visual realism and low-latency performance over the complex computations found in traditional numerical solvers.

2.4 Particle systems

2.4.1 Real-time particle systems

Within computer graphics applications, volcanic plumes can be simulated utilizing particle systems, an approach similar to the representation of other atmospheric phenomena such as smoke, fog, and clouds (Rautenhaus et al., 2017; Engel et al., 2004; Akenine-Moller et al., 2019). Real-time (RT) particle systems model these phenomena as collections of "fuzzy objects" (Reeves, 1998) that lack well-defined surface, distinguishing them from traditional 3D models composed of explicit polygonal surfaces. Every constituent particle is dynamic, continuously altering its morphology and position over a discrete lifespan that spans from its birth to its eventual dissipation.

Modern frameworks replicate the stochastic properties of individual particles

by exposing various parameters that users can adjust to mimic specific visual effects like fire, smoke, or fog. This methodology is formally classified as a Lagrangian perspective. Particles originate from an emitter, which serves as the spawning source within a 3D grid. The geometric configuration of the emitter plays a critical role in shaping the final visual output; for instance, a conical emitter is highly suitable for volcanic plume simulations because it mirrors the upward-expanding, conical trajectory characteristic of eruption columns.

Volumetric density is typically regulated via an "emission rate" parameter, while lifetime behaviors are dictated by variables governing velocity, color, and transparency transitions, or by enabling particles to dissolve upon intersecting solid geometry. Rendering is generally executed using simple primitives, such as points or lines, or via screen-aligned 2D textures known as billboards. In our implementation, we developed AnitoPlume to sample dynamically from a curated five-set texture library of smoke assets engineered to closely replicate the visual characteristics of an authentic volcanic plume.

Although RT particle systems are computationally efficient and highly capable of approximating plume behavior, they demand extensive manual parameter tuning (e.g., managing dimensions, emission frequencies, lifespans, velocities, and color gradients). Furthermore, they struggle to natively model key physical behaviors, such as the internal density distribution across the particle volume. Consequently, incorporating physical fluid dynamics becomes necessary to capture authentic plume mechanics (Section 2.5.2).

2.4.2 Particle systems and fluid dynamics

While real-time particle frameworks offer efficient approximations and excel at high-frame-rate visualization, they fundamentally lack the mechanism to resolve complex physical fluid interactions (Nishita & Dobashi, 2001). A volcanic plume represents a complex, multi-phase mixture of gases and ash; standard real-time particle systems cannot autonomously compute environmental interactions involving atmospheric clouds, wind fields, air entrainment, and gravitational forces (Lastic et al., 2022; Pretorius et al., 2024). To resolve this limitation, several methodologies integrate fluid solvers to govern underlying particle motion, which is subsequently visualized via a real-time engine or exported to offline production suites like Houdini (Fedkiw, 2003; Pfaff, Thuerey, & Gross, 2012; Lastic et al., 2022; Pretorius et al., 2024).

Purely Lagrangian frameworks utilized in RT graphics incur steep computational overhead when managing massive particle counts ($>$ 500K on mid-range

graphics hardware), rendering them impractical for highly interactive, real-time applications. To address this, researchers leverage Eulerian architectures, where grid-based solvers prove more efficient for representing continuous media like fluids and capturing high-fidelity scientific details (Jänicke & Scheuermann, 2010; Pfaff et al., 2012; Saidi, Rismanian, Monjezi, Zendehbad, & Fatehiboroujeni, 2014). Hybrid Eulerian–Lagrangian systems have also been introduced to exploit the dual advantages of both paradigms, with recent implementations deployed in cloud and cyclone modeling (Vimont et al., 2020; Stam, 2023; Pretorius et al., 2024; Amador Herrera et al., 2024). To contextualize this, Figure 5 outlines different kinds of particle-based modeling:

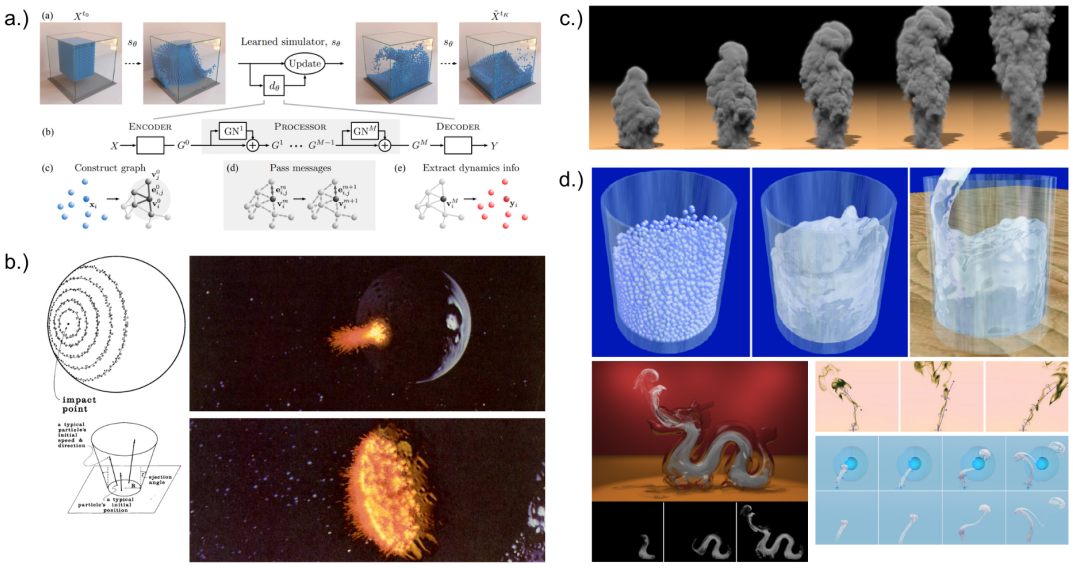

Figure 5: Overview of different particle systems and fluid simulations. From top to bottom; left to right: graph neural network (GNN) for physics simulations, procedural particle system for rendering fuzzy physical phenomena, high-resolution time evolution of a smoke explosion, compilation of different particle simulations.(Sanchez-Gonzalez et al., 2020; Reeves, 1998; Selle et al., 2005; Nowrouzezahrai, 2006).

The core simulation pipeline of AnitoPlume closely mirrors the Lagrangian volcanic plume model developed by Lastic et al.. Within this framework, column dynamics are resolved via a top-down architectural approach: an ascending circular cross-section simulates the rising eruption column emerging from the volcanic crater, which subsequently spawns rotating spheres for each discrete layer. This rotational behavior explicitly accounts for the localized vorticity and plume rotation documented in physical volcanic events (Nabizadeh, Roy-Chowdhury, Yin,

Ramamoorthi, & Chern, 2024; Braun, Bender, & Thuerey, 2025). In modernizing the application's graphics API, these rotating spheres are translated into instanced, screen-aligned billboard textures, which are dynamically sampled from the system's five-set smoke asset library based on the particle's instantaneous altitude.

2.5 Terrain rendering

Virtual terrains are typically represented through digital elevation models (DEM), such as textured meshes or heightmaps. Approaches to constructing these terrains can generally be categorized by how the underlying elevation data is obtained. *Acquisition-based techniques* derive terrain data from real-world sources, such as satellite imagery (e.g., Google Maps/Earth) or LiDAR scanning, allowing the reconstruction of terrain that reflects actual geographical landscapes. In contrast, *procedural techniques* generate terrain algorithmically, typically through noise-based functions, without reliance on real-world source data. Regardless of acquisition method, elevation data is commonly represented as a *heightmap*, a widely adopted format consisting of a 2D raster grid of elevation values, which is utilized to extrude a flat plane into a three-dimensional surface.

When modeling terrains inspired by actual geographical landscapes, data must be sourced from geographic information systems, including satellite imagery or LiDAR maps. Aside from ground elevation, raw LiDAR scans also include reflections from vegetation, structures, and other above-ground features that must be filtered out before a usable terrain surface can be extracted. Figure 6 illustrates this filtering process, showing the progression from raw above-ground height measurements to an isolated Bare Earth DEM suitable for terrain extrusion.

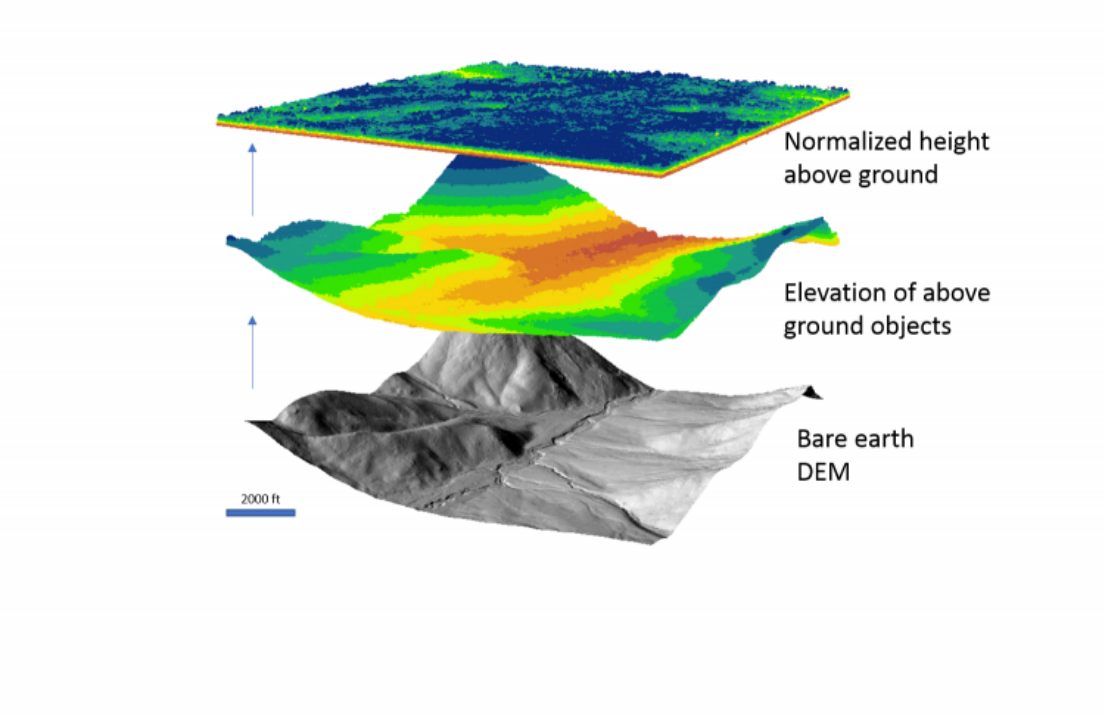

Figure 6: Example of processed LiDAR output layers. From (U.S. Geological Survey, 2018).

While converting satellite and LiDAR data into a heightmap to generate a 3D terrain within visualization software is a direct methodology, it remains susceptible to data gaps and structural inaccuracies, particularly when the underlying DEMs are captured by low-resolution sensors.

Prior works focus on the volcanic eruption behaviors, such as lava propagation or plume dynamics, while rarely accounting for volcanic topography during the simulation process. Because AnitoPlume was designed as a holistic 3D simulator of the Taal Volcano, terrain rendering constitutes a core component of its architectural framework. The baseline 3D model of the Taal Volcano Island was constructed using an acquisition-based approach: LiDAR data was extracted from opentopography.org (openTopography, 2025) and processed within QGIS (Moyroud & Portet, 2018), a geographic information system utilized to generate the requisite heightmap. This heightmap was leveraged to produce an initial 3D mesh of Taal, which was subsequently refined using Blender (Blender Foundation, 2025). Any incomplete segments within the original LiDAR data were manually resolved using digital inpainting, sculpting, and smoothing workflows. Figure 7 summarizes this end-to-end pipeline, from LiDAR-derived heightmap conversion

in QGIS through mesh refinement in Blender.

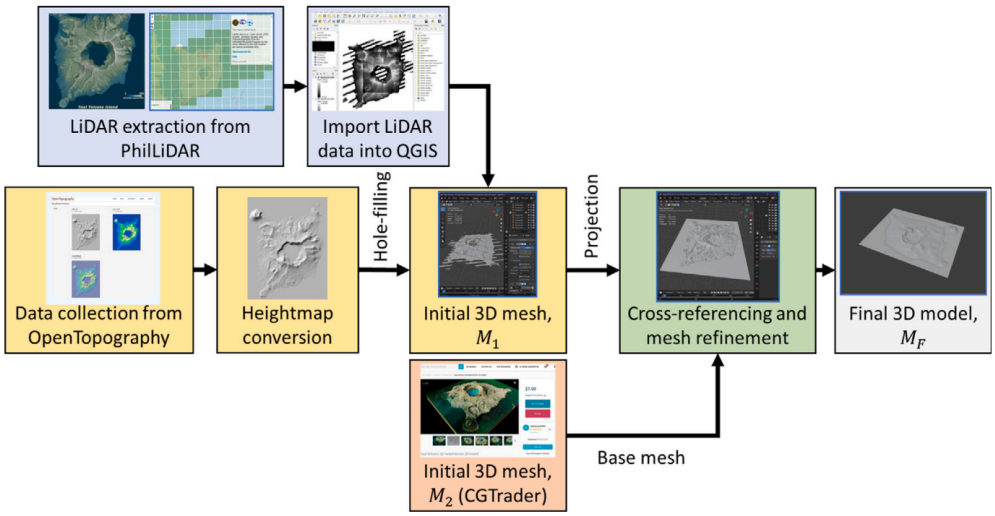

Figure 7: AnitoPlume's terrain modeling workflow of Taal Volcano. From (Del Gallego et al., 2026).

2.6 Volcanic simulations

Some computer graphics researchers focus on the generation of visually convincing lava flow animations. For instance, Zhang et al. created a hybrid framework engineered to simulate lava viscosity while accounting for thermal transfer and surface friction mechanics. To capture granular-fluid flows—which are highly applicable to mass-wasting events such as landslides, avalanches, and lava progression—researchers frequently adopt debris flow models because they synthesize foundational concepts from fluid, solid, and soil mechanics (Iverson & George, 2014; George & Iverson, 2014; Trujillo-Vela, Ramos-Cañon, Escobar-Vargas, & Galindo-Torres, 2022).

As for the simulation of volcanic plumes, it typically employs the use of atmospheric transport or scattering models, ensuring that the simulated plume interacts realistically with the ambient air or wind (Lastic et al., 2022; Pretorius et al., 2024). New novel fluid such as Coadjoint Orbit FLIP (CoFLIP) (Nabizadeh et al., 2024) and Adaptive-FLIP (Braun et al., 2025), both of which operate as modern extensions of fluid implicit particle (FLIP) formulation (Brackbill & Ruppel, 1986). Among these developments, the framework proposed by Lastic et al. represents the first work observed so far that can perform real-time plume simu-

lation, due to utilizing a streamlined Lagrangian simulation. Building upon this, Pretorius et al. proposed a coupling model of explosive eruptive behaviors directly with atmospheric simulations to render volcanic skyscapes.

Figure 8 illustrates representative outputs from real-time plume simulation frameworks, contrasting the Lagrangian-based approach of Lastic et al. and the coupled skyscape model of Pretorius et al. with AnitoPlume's own real-time volcanic plume simulation of Taal Volcano, which serves as the baseline system extended in this work.

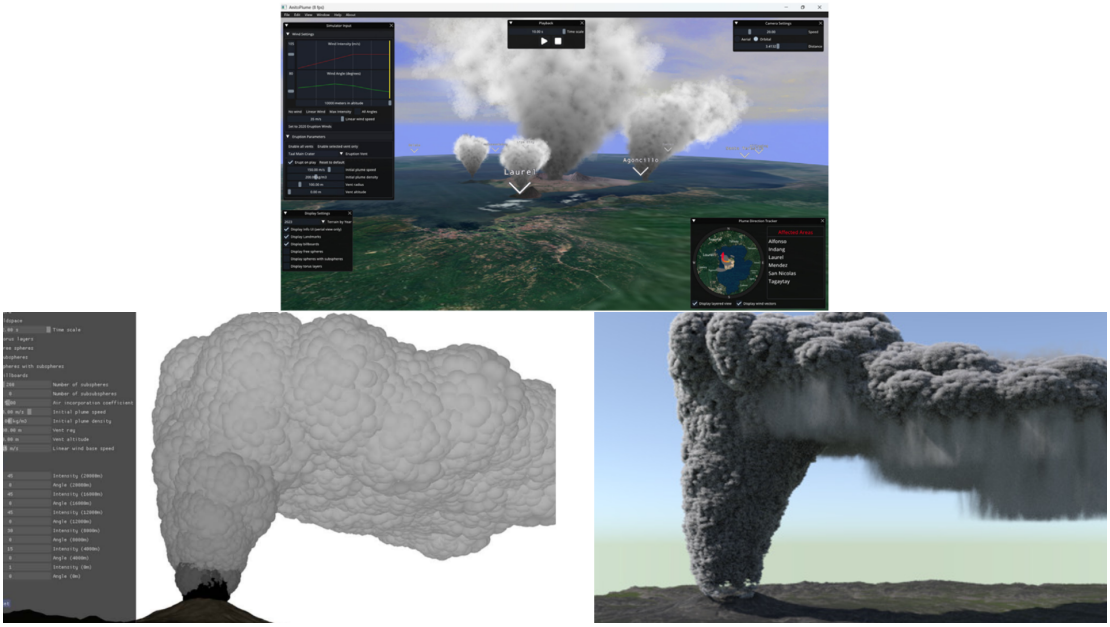

Figure 8: Volcanic Simulations from (Del Gallego et al., 2026) (top), (Lastic et al., 2022) (bottom-left) and (Pretorius et al., 2024) (bottom-right).

Alternative domains of research prioritize the analysis and interpretation of eruption data (Pardini et al., 2024). Recently, several works has expanded to include deep learning and semantic segmentation techniques designed to automatically detect and isolate volcanic plumes within optical camera imagery (Wilkes, Pering, & McGonigle, 2022; Guerrero Tello, Coltelli, Marsella, Celauro, & Palenzuela Baena, 2022; Torrisi, Corradino, Cariello, & Del Negro, 2024; Swetha, Datla, & Vishnu, 2023).

2.7 AnitoPlume

AnitoPlume is the first interactive, real-time 3D graphics-based simulation application explicitly tailored for visualizing the changing landscape of the Taal Volcano island in the Philippines and its corresponding eruption plumes. Developed primarily to address the challenges of volcanic hazard and crisis communication, the simulator serves as an immersive, lightweight alternative to traditional 2D heatmaps and highly complex, non-interactive geoscientific numerical solvers (such as FALL3D and Tephra2). While geoscientific solvers excel at quantitative predictive accuracy, they require heavy high-performance computing resources and significant processing time, making them unviable for applications requiring immediate visual feedback, interactive flexibility, and low-latency user immersion. AnitoPlume fills this gap by merging computer graphics techniques with site-specific real-world terrain details to facilitate risk awareness and "what-if" situational modeling (Del Gallego et al., 2026). To visually demonstrate these differences in rendering approach and graphical fidelity, Table 1 compares the output of previous plume simulations with the real-time terrain integration of our proposed simulator.

	Geoscientific Models (e.g., FALL3D, Tephra2)	**Computer Graphics Related Work (e.g., Lastic et al. (2022))**	**AnitoPlume (Proposed Simulator)**
Method	Numerical solvers, Eulerian atmospheric dispersion models, Tephra accumulation and dispersion.	Eulerian, Lagrangian, or hybrid particle systems approximation.	Lagrangian system with synchronized multi-vent plume logic.
Visualization	Static 2D/3D low-fidelity contours, graph-based and/or scientific visualization, heatmaps.	3D scene using simple spheres, billboard particles, static planes, or basic landscapes.	3D scene using LiDAR and OpenTopography terrain data, dynamic textures, textured particles, interactive UI with plume trackers.
Performance	High accuracy at the cost of high computation time.	Real-time performance prioritized for visual realism. Approximation of physical eruption behavior.	Real-time performance with improved visual realism. Optimized for 30 FPS. E.g., Terrain-specific model through multi-source 3D modeling workflow.
Temporal Representation	Focuses on snapshots of single eruptive events.	Continuous loop simulations with emphasis on plume structure alone.	Dynamic textures: Multi-year landscape visualization (2015–2023), different plume particles based on duration.
User Interface (UI)	Displays comprehensive outputs and calculations. E.g., dispersal, mass accumulation, probability maps.	Parameter tuning for visual and physics adjustments, either via the UI or via text/command-line input.	A user application requiring no text prompt. Revised layout for parameter tuning. UI is suited for "what-if" scenarios because of integrated hazard trackers.
Validation Strategy	Field-constrained analysis of tephra fall deposits.	Qualitative visual comparisons	Quantitative similarity analysis — structural image similarity (SSIM) and root mean squared error (RMSE). Usability testing conducted.

Table 1: Comparison of previous plume simulation (left) implement by Lastic et al. (2022) and geoscientific numerical solvers (middle) with the AnitoPlume simulator (right) by Del Gallego et al. (2026).

2.7.1 Graphics Architecture and System Performance

To bypass the overhead, heavy abstractions, and unneeded general-purpose features of commercial game engines, AnitoPlume was custom-built using a lean, native C++ open-source API stack consisting of OpenGL for graphics rendering, Eigen for numerical linear algebra, and dearIMGUI for the user interface layout. This custom application architecture optimizes real-time performance on consumer-grade hardware. By keeping the system dependencies minimal, the simulator successfully achieves real-time interactivity, running at an average rendering rate of 30 frames per second (FPS) on a system with 6 GB of video RAM (VRAM) capped at 2080 x 2080 resolution(Del Gallego et al., 2026).

2.7.2 Usability Design and Interactive Features

The simulator also introduced several usability improvements over previous volcanic visualization systems, including simplified camera controls, interactive parameter editing, real-time plume direction tracking, and contextual information about nearby communities affected by volcanic activity. These additions improved both accessibility and user experience while preserving interactive performance (Del Gallego et al., 2026).

2.7.3 System Validation and Evaluation Metrics

To evaluate the effectiveness of the simulator, Del Gallego et al. conducted quantitative and qualitative assessments. Visual fidelity was measured using the Structural Similarity Index (SSIM) and Root Mean Squared Error (RMSE) by comparing simulated eruption images with real-world photographs and surveillance footage of Taal Volcano. The study reported high visual similarity, indicating that the synthesized plume closely resembled actual volcanic eruptions. In addition, usability testing using the System Usability Scale (SUS) demonstrated that participants generally found the simulator easy to use and suitable for interactive exploration.

2.7.4 Architectural Limitations and Future Trajectories

Despite these accomplishments, the primary contribution of AnitoPlume lies in the design of its visualization framework rather than the underlying graphics API.

The study focused on plume simulation, terrain modeling, visual fidelity, and usability evaluation, while the OpenGL implementation served primarily as the rendering platform supporting these objectives. Consequently, the original work did not investigate how the rendering architecture itself might influence performance, resource utilization, maintainability, or future extensibility.

The authors also acknowledged several opportunities for future development, which include:

1. Photorealistic Rendering
2. Expanded Physics-Based Mechanics
3. Image-Based Parameter Estimation
4. Geoscience Data Integration
5. Automated Terrain Synthesis

These recommendations highlight the critical need for a modern rendering architecture and a low-level API like DirectX 12, which are capable of supporting such intensive graphics and parallel computing workloads while maintaining real-time interactive performance.

2.8 API Migration in Rendering Systems

The migration of rendering software between graphics APIs is a well-known challenge in computer graphics. It often involves more than just replacing function calls. A successful port must account for differences in shader languages, resource-binding models, render-target handling, memory synchronization, and execution semantics. In practice, this means that a program cannot simply be translated line by line; instead, the rendering architecture must be reinterpreted in the context of the target API.

Previous work in graphics engineering has shown that API migration is most successful when the original system is structured in a modular way. Systems that separate rendering logic, scene management, and platform-specific code are often easier to port than tightly coupled implementations. This is particularly relevant for research software, where maintainability and clarity are essential. The process of porting from OpenGL to DirectX, therefore, becomes not only a technical exercise, but also an opportunity to improve the software architecture.

2.8.1 Cross-API Shader Translation

A major issue in porting visual effects from OpenGL to DirectX 12 is shader translation. OpenGL workflows usually rely on the OpenGL Shading Language (GLSL), while DirectX 12 expects the High-Level Shader Language (HLSL) and a more structured offline compilation pipeline. Existing developer practice tends to fall into two categories: manual shader rewriting for tight control over hardware-specific optimizations, or cross-compilation workflows that preserve shader intent while reducing rewrite cost (Capodieci, Cavicchioli, & Marongiu, 2022).

For a visually sensitive effect such as AnitoPlume's volcanic plumes, the second approach is highly attractive. Manual translation risks introducing numerical drift and subtle visual discrepancies in lighting and blending logic. This design philosophy is exemplified by the broader shift in modern graphics tooling toward intermediate shader representations, which architecturally separate program descriptions from backend compilation. By utilizing intermediate representations like SPIR-V, such tooling prioritizes shader portability over hard-coded backend assumptions. In a migration study, this supports the argument that accurate visual porting is not just about translating syntax from GLSL to HLSL, but about maintaining shader behavior across compilation targets using tool-assisted pipelines like the DirectX Shader Compiler (DXC) or SPIRV-Cross (Kinkor, 2022).

2.8.2 Pipeline State Management Adaptation

A second major challenge is the shift from OpenGL's dynamic state model to DirectX 12's explicit Pipeline State Objects (PSOs). In architectures, OpenGL allows rendering states to be changed incrementally at runtime, with the graphics driver silently inferring and tracking state combinations (J. A. Shiraef, 2016). DirectX 12 completely eliminates this hidden driver-side management. Instead, explicit APIs require developers to bake the blend state, rasterizer state, depth-stencil state, primitive topology, and shader kernels into immutable PSOs ahead of time (Bossard, 2019).

The necessity of this structural rewrite is inherent to the explicit PSO model itself. Rather than relying on hidden driver-side state inference, explicit graphics APIs require applications to aggregate vertex layouts, shader kernels, rasterizer states, and framebuffer descriptions into a single, pre-compiled state object definition ahead of time. At draw time, the render context binds this monolithic object directly before issuing the call. For plume rendering, adopting this explicit design is critical; even minor mismatches in depth testing or alpha blending across different PSOs can fundamentally alter how overlapping particle billboards accumulate

and fade (Del Gallego et al., 2026). Therefore, PSO migration must be framed as a foundational architectural overhaul rather than a simple API substitution (Asplund, 2020).

2.8.3 Resource Synchronization and Memory Barriers

The most technically demanding aspect of the explicit migration process is resource synchronization. In OpenGL, hardware hazards are concealed by the driver, meaning developers rarely need to explicitly manage the transitions between writing to and reading from the same texture or buffer (Unterguggenberger et al., 2022). DirectX 12, by contrast, transfers the burden of memory management entirely to the developer, requiring explicit resource barriers to ensure that GPU accesses occur in the correct deterministic order (Rossant & Rougier, 2021).

This explicit synchronization burden is a defining characteristic of explicit graphics APIs, which require applications to issue resource barriers that route resources into their correct states before any updates, copies, or draw calls are executed. By manually handling transitions at both the resource and subresource levels, these architectures reinforce that state transitions are an application responsibility rather than an invisible driver service. For the AnitoPlume study, this supports a key argument: accurate migration is not just about producing a similar output image, but about guaranteeing that the GPU views each buffer in the correct state at the exact right microsecond. This is essential when updating thousands of particle billboards, as incorrect barrier placement during parallel draw calls leads to race conditions, stale texture reads, flickering artifacts, or catastrophic device crashes (Cheung, 2024).

Chapter 3

Computer Graphics and Rendering Concepts

The modernization of AnitoPlume involves migrating the rendering pipeline from a traditional to a modern graphics API architecture. This migration introduces a fundamentally different graphics architecture characterized by explicit resource management, reduced driver overhead, and improved utilization of modern multi-core processors. These architectural changes are beneficial for simulators such as AnitoPlume, which require continuous updates of thousands of dynamic particle billboards.

3.1 Graphics Rendering Architectures

Traditional graphics API architectures are designed to abstract low-level graphics operations from the programmer by implicitly managing graphics state, resource synchronization, and command validation within the graphics driver. Throughout this study, these are referred to as implicit graphics APIs, whereas architectures that expose these responsibilities directly to the application are referred to as explicit graphics APIs.

3.1.1 Overview of Graphics API Architectures

Implicit graphics APIs such as OpenGL are based on a state-based programming model where the application modifies global rendering states that are maintained

by the graphics implementation ("The OpenGL Graphics System: A Specification", 2022). Implicit graphics API architecture relies on a thick driver layer that abstracts graphics pipeline management from the application (Fig. 1). Because the driver is responsible for maintaining graphics state, performing runtime validation, tracking resource usage, and managing command execution, significant CPU overhead is introduced during rendering (Ping et al., 2026).

Explicit graphics architectures expose synchronization, resource management, and command submission responsibilities directly to applications (Khronos Group, 2024). Rather than relying on implicit driver-managed state changes, explicit graphics architectures require the application to explicitly define and manage the configuration of the graphics pipeline. Applications explicitly define pipeline states, resource allocation, synchronization, and memory management as seen on figure 9.

OpenGL	**Vulkan**	**DirectX 12**
API originally created for graphics workstations; uses split memory.	Match for modern platforms (mobile/unified memory/tiled rendering).	Modern API for Windows/Xbox; scales across unified platforms.
Driver handles state validation/error tracking, limiting performance.	Explicit API gives application direct, predictable GPU control.	Shifting validation to driver, giving developers explicit resource control.
Obsolete single-threaded model; no parallel command generation.	Built for multi-core/threads; parallel command buffers.	Multi-core optimized; pre-computes command lists on any CPU core.
Complex choices; syntax evolved over 20+ years.	No legacy requirements; simpler API design and smaller specification.	Replaces legacy state objects with unified Pipeline State Objects (PSO).
In-driver GLSL compiler; requires shipping raw shader source.	SPIR-V target enables front-end language flexibility and reliability.	Compiles HLSL to intermediate DXIL/DXBC to reduce driver workload.
High vendor-specific implementation variability.	Simpler API and testing improve cross-vendor compatibility.	Strict Feature Levels (e.g., DX12 Ultimate) ensure vendor consistency.

Table 2: Graphics API Comparison: OpenGL vs. Vulkan vs. DirectX 12 from J. Shiraef and Zoraja et al.

Table 2 summarizes the key architectural and performance differences between the OpenGL, Vulkan, and DirectX 12 graphics APIs. It contrasts the single-

threaded design of OpenGL with the modern, multi-threaded, and explicitly controlled paradigms introduced by Vulkan and DirectX 12.

The limitations of the traditional graphics API architecture become particularly significant in computationally intensive scientific visualization systems such as AnitoPlume. In addition to numerically integrating volcanic plume dynamics including gravity, buoyancy, drag, and wind forces for thousands of particles each simulation step, the CPU must also handle graphics driver validation and state management (Del Gallego et al., 2026). This added overhead limits rendering performance, motivating the use of lower-overhead graphics APIs to reduce CPU-side workload for applications that require high-throughput rendering. As rendering workloads become more complex, these driver-managed operations can limit the application's ability to optimize resource usage, creating CPU-side bottlenecks that reduce the rate at which work can be submitted to the GPU (Ping et al., 2026).

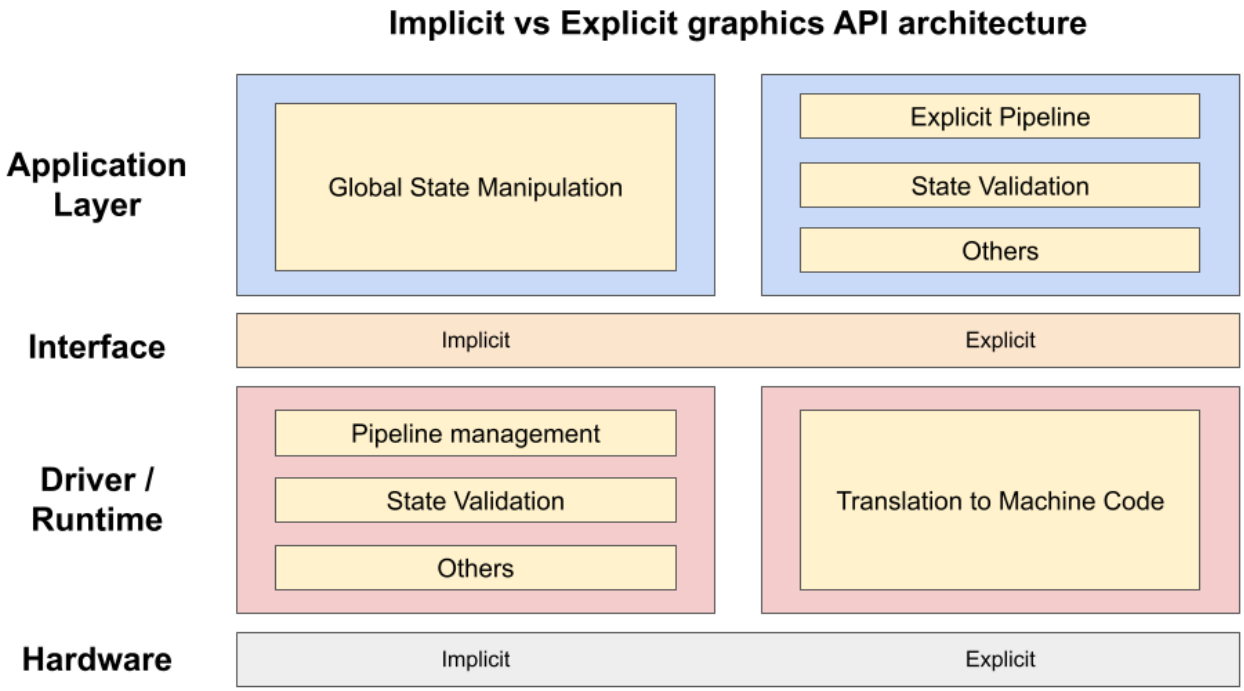

Figure 9: Comparison of implicit graphics API architecture (left) with explicit graphics API architecture (right).

3.1.2 Performance Optimization opportunities

Explicit graphics architectures reduce driver-managed serialization by shifting the responsibility to the application. Tasks such as pipeline configuration, resource allocation, and command preparation can be performed explicitly by the application before or outside the main rendering loop. During runtime, the system only executes predefined instructions without modifying or accessing the underlying data. This shift allows command generation to be safely distributed across multiple CPU threads (Ping et al., 2026). The lightweight driver architecture introduced by explicit graphics APIs minimizes runtime validation and hidden state management, reducing CPU overhead while providing developers with greater control over rendering operations. By enabling application-driven optimization and more efficient utilization of hardware resources, explicit graphics API architectures are better suited for performance-critical rendering workloads (Ioannidis & Boutsi, 2020).

3.2 Lagrangian and Eulerian Models for Volumetric Plume Dynamics

3.2.1 Lagrangian Formulation

AnitoPlume utilizes a Lagrangian formulation for representing volcanic plume evolution, where the plume is modeled as a collection of discrete volumetric elements whose physical states are updated over time. Unlike Eulerian approaches that evaluate flow properties within fixed spatial domains, Lagrangian approaches track the movement and evolution of individual elements within the flow field (Esposti Ongaro et al., 2020).

The plume formulation presented by Lastic et al. (2022) and implemented in AnitoPlume (Del Gallego et al., 2026) represents plume dynamics through discrete elements with associated physical properties. Each plume element maintains state variables such as position, velocity, mass, density, and volume. During each simulation step, these variables are updated based on governing physical equations that consider forces including gravity, buoyancy, atmospheric drag, and wind effects (Lastic et al., 2022; Del Gallego et al., 2026; Folch, Costa, & Macedonio, 2016).

The structure of the Lagrangian formulation exhibits data-parallel characteristics because individual plume elements can be updated primarily using their local

state information and shared simulation parameters. Consequently, operations such as force evaluation and numerical integration can be distributed across multiple processing units, provided that synchronization requirements are properly managed (Lastic et al., 2022).

However, Lagrangian particle-based methods also present limitations when applied to large-scale geophysical phenomena. The computational cost associated with representing the large number of particles required for realistic volcanic processes has limited the widespread use of fully Lagrangian approaches in volcanological simulations (Esposti Ongaro et al., 2020). These limitations motivate the consideration of alternative formulations, model refinements, or modifications to the underlying mathematical representation depending on the accuracy and computational requirements of the simulation. Consequently, the selection of governing equations and the formulation of the plume model remain important considerations in the continued development of volcanic simulation systems.

Despite these considerations, the data-parallel characteristics of the Lagrangian representation make plume simulation suitable for optimization through modern parallel processing architectures. Since the computational workload consists of continuously updating large numbers of dynamic elements, improving execution efficiency requires an architecture capable of reducing overhead and efficiently utilizing available hardware resources. Explicit graphics architectures provide mechanisms that support this objective by enabling greater control over resource management and parallel command execution.

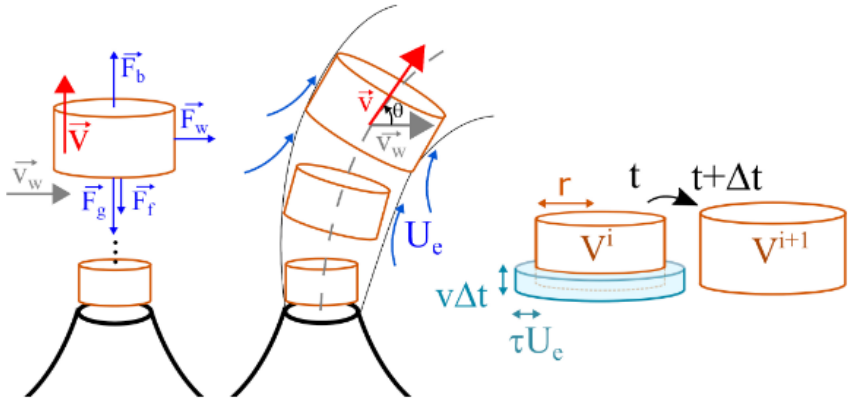

Figure 10: Lagrangian Approach (from (Lastic et al., 2022)).

Figure 10 illustrates the Lagrangian formulation implemented by (Lastic et al., 2022), detailing the physical forces, wind-driven trajectory changes, and volumetric expansion acting on individual discrete plume elements over time.

3.2.2 Eulerian Formulation

In contrast to the Lagrangian approach, the Eulerian formulation describes plume dynamics with reference to fixed points or cells in space, rather than tracking discrete elements as they move through the flow field. Under this formulation, the volcanic plume is represented as a continuous field of quantities, such as ash concentration, density, temperature, and velocity, which are computed and updated at fixed grid locations over time as material flows through them (Esposti Ongaro et al., 2020). Rather than following individual particles or plume elements as they travel through the atmosphere, the Eulerian approach numerically solves governing transport equations directly on a spatial grid that discretizes the simulation domain. This formulation underlies many of the numerical geoscientific models used for volcanic hazard forecasting; for instance, models such as FALL3D use Eulerian atmospheric dispersion techniques to compute the transport, dispersion, and eventual deposition of volcanic tephra across a defined spatial domain (Folch et al., 2009; Del Gallego et al., 2026). While this grid-based approach is generally well-suited for capturing diffusive processes and producing quantitatively accurate forecasts, it comes at a significant computational cost, particularly as grid resolution increases or the simulation domain grows, since the governing equations must be solved at every grid cell regardless of whether volcanic material is present at that location. Consequently, Eulerian volcanic transport models are typically designed for offline scientific analysis and hazard mapping rather than for real-time or interactive visualization (Del Gallego et al., 2026).

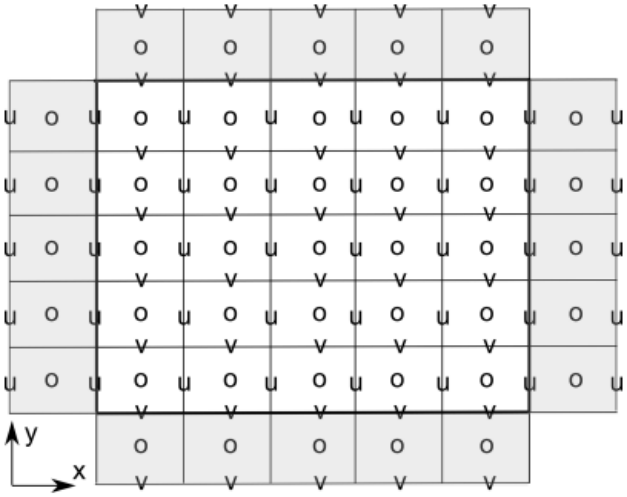

Figure 11: Eulerian Approach (from (Folch et al., 2020)).

For the Eulerian Approach, Figure 11 visualizes how it discretizes the simula-

tion space into a 3D staggered mesh of fixed computational cells to evaluate mass transport and flow velocity over time.

3.2.3 Comparison of Both Approaches

Table 3: Summary of key theoretical distinctions between the Lagrangian and Eulerian formulations.

Dimension	Lagrangian Formulation	Eulerian Formulation
Frame of reference	Tracks discrete, moving plume elements.	Evaluates fixed spatial grid points.
Representation	Discrete particles with individual states (pos, vel, mass).	Continuous field of quantities (concentration, density).
Computation	Scales with particle count; local-state updates.	Scales with grid size; solved at every cell.
Accuracy	Approximates diffusive behavior.	High quantitative accuracy for diffusive transport.
Real-time	Well-suited; lightweight for moderate counts.	Generally unsuitable; computationally demanding.
Representative use	Lastic et al. (2022); AnitoPlume	FALL3D (Folch et al., 2009)
Limitation	Cost grows with particle count for visual realism.	Cost grows with grid resolution/domain coverage.

AnitoPlume adopts the Lagrangian formulation because it prioritizes real-time interactivity over the grid-based accuracy of Eulerian models. This study preserves that formulation as a fixed theoretical foundation, since its scope is limited to the rendering and software architecture layer, not the underlying plume dynamics. The Lagrangian model's data-parallel characteristics, individual elements updating largely independently using local state, are what make it a suitable target for the proposed low-level rendering architecture, which is designed to expose fine-grained control over parallel command execution and GPU resource management. This theoretical basis justifies why the present study's contributions are directed at the rendering pipeline rather than the plume simulation model itself.

Chapter 4

Methodology

The primary objective of this research is to port and extend AnitoPlume's real-time simulation framework by modernizing its rendering pipeline, optimizing hardware utilization through new, explicit graphics APIs, and refining its underlying physical simulation models.

4.1 Implementation and Migration

4.1.1 Legacy Simulation

The legacy execution architecture relies on a strictly sequential, single-threaded pipeline driven by an iterative time-stepping loop. Within each frame slice, the simulation runs a high-frequency sub-stepping loop (executing ten physics updates per render frame) to remove visual smoke jumping and oscillations due to numerical instability. The control flow inside this sub-step operates linearly across distinct global entity arrays.

As shown in Figure 12, the simulation is organized around the smoke layer as the primary simulation unit. Each smoke layer models the behavior of the eruption column by computing atmospheric air entrainment, thermodynamic properties (e.g., temperature and density), external forces such as buoyancy, gravity, wind, and drag, and integrating the resulting motion over time. Attached to every smoke layer are free spheres, which represent smoke that inherit the motion and physical state of their parent layer while evaluating transitions into falling or neutrally buoyant states. Each free sphere further generates a hierarchy of subspheres that

provide the volumetric detail required for visualization. These subspheres are rendered as billboard sprites to produce the final volumetric appearance of the smoke cloud but do not participate in the physical simulation.

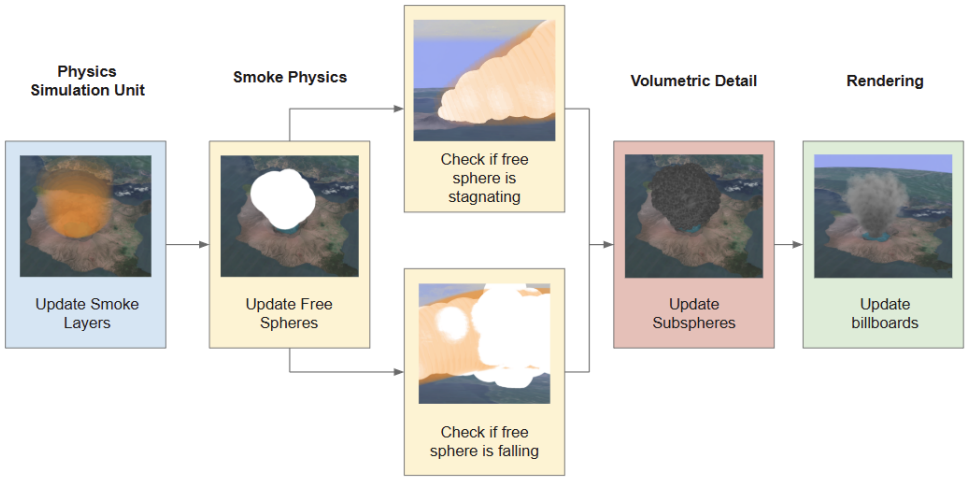

Figure 12: Legacy Physics Simulation Loop.

AnitoPlume initializes the simulation environment by loading a preprocessed three-dimensional terrain model representing Taal. The terrain mesh is generated offline from geospatial elevation datasets and optimized for interactive rendering while preserving the topographic features required for volcanic hazard visualization (Del Gallego et al., 2026). Textures for the terrain model were made through photobashing of satellite images taken from Google Earth; as seen on Figure 13 (Del Gallego et al., 2026).

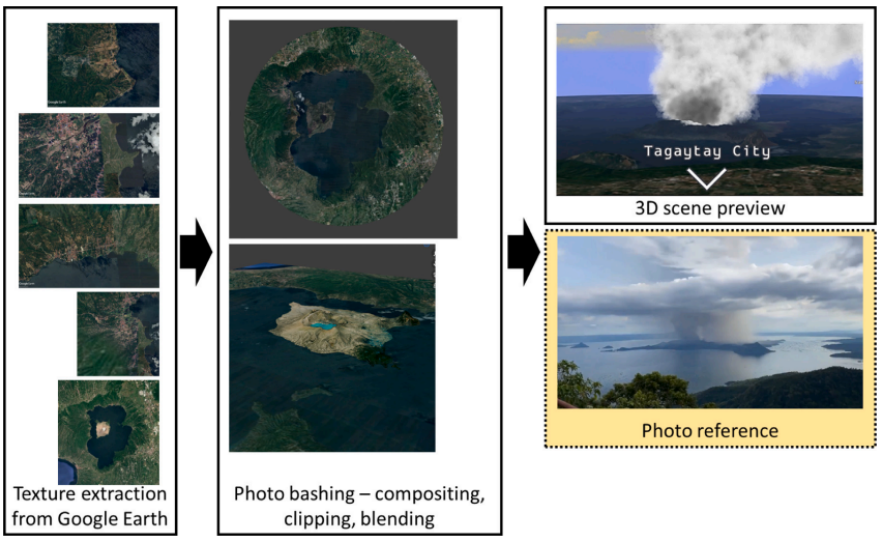

Figure 13: Workflow for texturing the surrounding area of Taal Volcano. From (Del Gallego et al., 2026).

During initialization, the terrain is imported into the rendering pipeline, transformed into the simulation coordinate system, and associated with its corresponding surface texture.

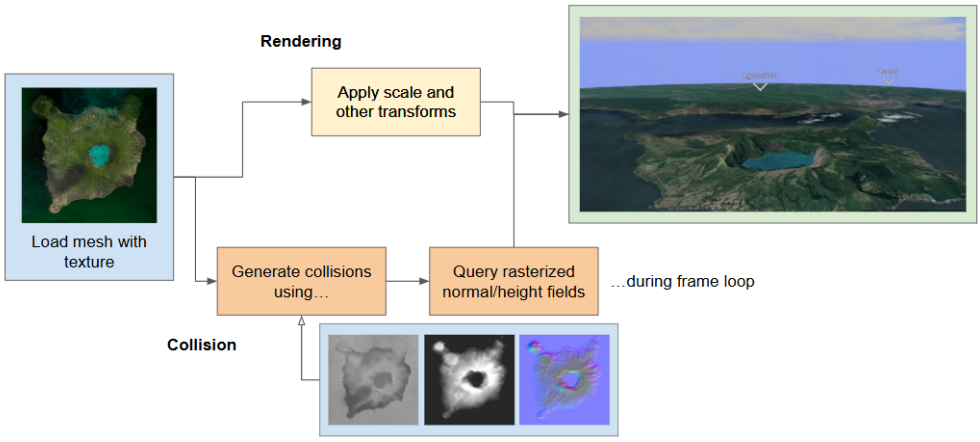

Figure 14: Legacy Terrain Rendering PipeLine.

In addition to visualization, the terrain serves as the physical boundary of the simulation. Height and surface normal information are queried during particle

updates to detect terrain intersections and compute particle-ground interactions, allowing falling volcanic material to respond to the underlying topography. Consequently, the terrain functions as both the rendered environment and the collision surface used by the simulation.

Beyond the core simulation and terrain rendering pipelines, AnitoPlume also provides an interactive graphical user interface (GUI) through which users can manipulate plume behavior in real time. This interface additionally surfaces contextual 3D tooltips that display relevant information about Taal Volcano directly within the scene. Figure 15 shows this interface, representing the primary means by which users interact with and observe the legacy simulation system.

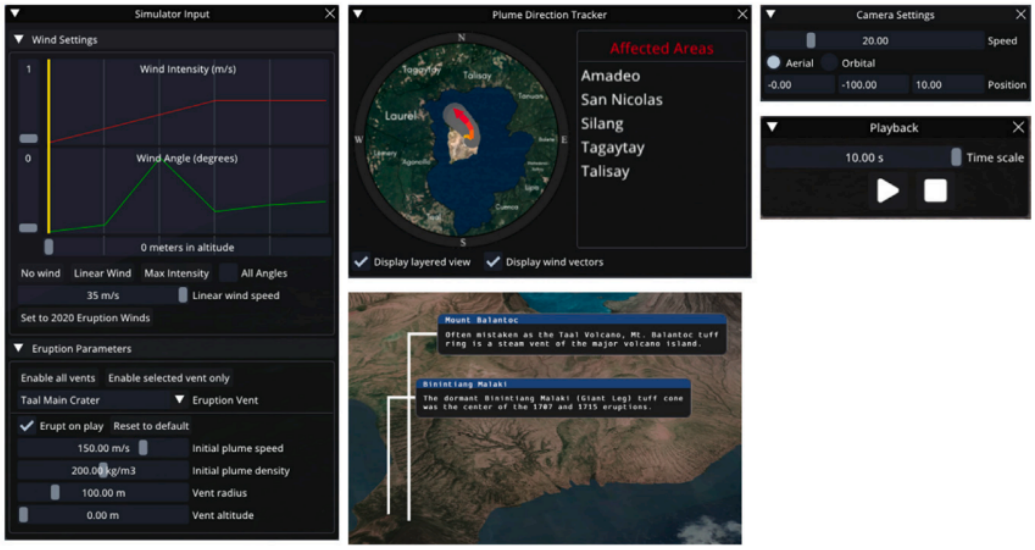

Figure 15: GUI of AnitoPlume for manipulating plume behavior and 3D UI tooltips for displaying taal information to the user. From (Del Gallego et al., 2026).

4.1.2 Parallelization of the Simulation Loop

One possible improvement for AnitoPlume's performance is to explore multithreaded rendering. To achieve this, we propose implementing parallelization to improve the responsiveness of the interactive simulation for multicore CPUs. This concurrent execution is feasible because individual smoke layers operate in complete isolation from one another. The mathematical functions executed during the smoke layer update phase do not feature any cross-layer spatial dependencies; instead, the physics equations for any given layer depend strictly on a shared set of initial environmental parameters, vent parameters, and global trackers. Because

later simulation stages consume data produced by earlier stages, we propose introducing barrier synchronization between execution phases, as shown in Figure 16. These synchronization points will guarantee that all worker threads complete one phase before subsequent phases begin, ensuring that rendering is performed only after all simulation stages have concluded to maintain visual consistency.

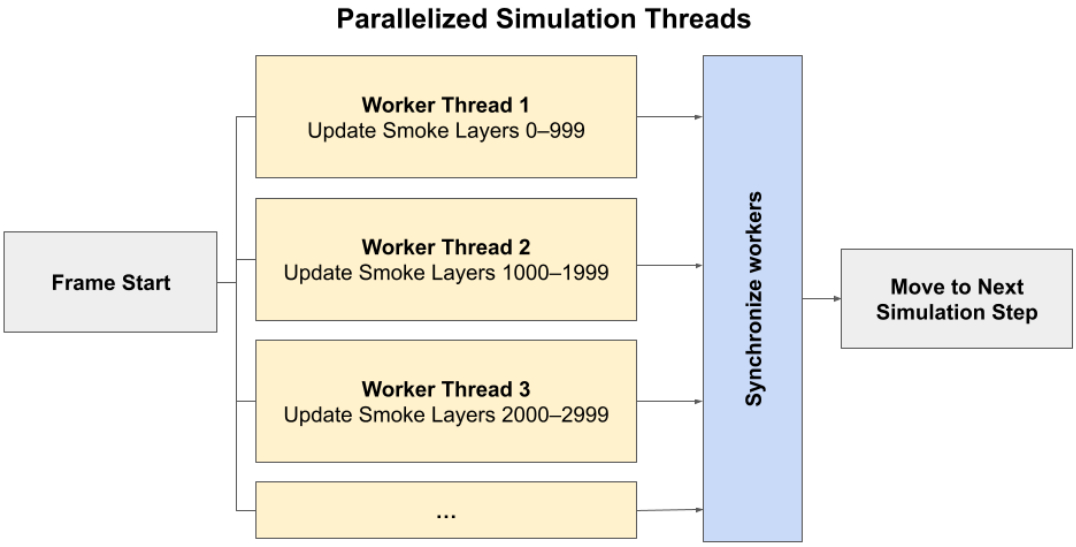

Figure 16: Parallelization of Smoke Layer Update Phase.

4.1.3 Terrain Rendering Improvements

While the terrain geometry will accurately capture the topography of the study area, the initial rendering implementation will be intentionally simple. We will render the terrain using a single textured mesh with basic diffuse lighting and fixed material properties, without terrain-specific rendering techniques such as multi-layer texture blending, physically based shading, normal mapping, ambient occlusion, or level-of-detail management.

The proposed rendering architecture will enable more advanced rendering techniques to be incorporated without modifying the underlying physics simulation, since we will manage rendering resources independently through explicit GPU resource management. One promising approach is to utilize this architecture to facilitate the integration of enhancements such as higher-resolution terrain textures, physically based materials, dynamic lighting, normal mapping, shadow mapping, terrain tessellation, and level-of-detail rendering while preserving the existing sim-

ulation workflow.

Although we will modernize the terrain rendering pipeline, the existing graphical user interface as described in Figure 2 will remain intact. We will preserve user interactions for terrain visualization, simulation control, and parameter configuration, ensuring compatibility with the original application workflow.

4.2 Benchmarking Procedure

Benchmarking will quantify the performance gains of the proposed architecture relative to the AnitoPlumeV1 baseline, using identical scenarios and hardware. For each scenario, the arithmetic mean of all runs will be used to reduce the effect of system variability, with results compared via absolute values and percentage improvement.

To evaluate scalability, performance metrics will be collected through increasing simulation workloads, using profiling tools such as RenderDoc to capture low-level pipeline data alongside application-level timing instrumentation. The specific metrics utilized for this evaluation, along with their respective descriptions, are detailed in Table 4.

Table 4: Performance metrics and descriptions used for evaluation.

Metric	**Description**
Average FPS	Overall rendering performance
Frame Time (ms)	Total frame execution time
CPU Frame Time	CPU workload
GPU Frame Time	GPU workload
Command Recording Time	CPU time spent preparing GPU commands
Shader Execution Time	GPU shader workload
GPU Memory Bandwidth	Memory utilization
CPU Utilization	Effectiveness of multithreading

Workload size and rendering performance will be visualized using line graphs illustrating trends in average frame rate, frame time, CPU utilization, and GPU utilization. GPU profiling metrics, including shader execution time, command recording time, and memory bandwidth utilization, will likewise be compared to determine how efficiently the proposed architecture utilizes available hardware resources.

Finally, the benchmarking results will be interpreted alongside the User Accep-

tance Testing (UAT) results to determine whether improvements in computational performance correspond to improvements in perceived responsiveness and overall user experience.

4.3 Usability Testing

To verify that the architectural overhaul maintains or improves the interactive user experience, an exploratory usability test will be conducted with target beneficiaries:

- Participants: Recruitment will follow a tiered priority order. PHIVOLCS personnel will be the top priority given their domain expertise in volcanology and direct familiarity with Taal Volcano hazard assessment. Should this pool be insufficient, recruitment will expand to, in order: (1) individuals who have witnessed a Taal Volcano eruption firsthand, (2) enthusiasts with experience using volcano/geohazard simulation software, and (3) computer science students with a background in rendering or volcano-related coursework/projects. This staged recruitment reflects the tool's intended end-users while ensuring an adequate sample size, and informs the corresponding participant categories documented in the study's ethics clearance and consent materials.
- Setup: After screening and obtaining consent from the Informed Consent Form (ICF), participants will be given time to explore the interface.
- Task: Participants will use the software to recreate specific Taal Volcano eruption plumes, following a set of predefined scenarios.
- Survey: Users will complete the System Usability Scale (SUS) questionnaire to rate the software's ease of use.
- Feedback: A closing interview will gather suggestions for improving the interface and overall performance.

Table 5: Sample SUS questionnaire implemented by (Del Gallego et al., 2026) 1 — strongly disagree, 5 — strongly agree.

Code	Question
Q1	I think that I would use AnitoPlume frequently.
Q2	I found AnitoPlume unnecessarily complex.
Q3	I found AnitoPlume easy to use.
Q4	I think that I would need the support of a technical person to be able to effectively and efficiently use AnitoPlume.
Q5	I found that various features of the simulator were well integrated in AnitoPlume.
Q6	I thought there was too much inconsistency in AnitoPlume.
Q7	I would imagine that most people without prior experience in using similar software would easily learn how to use AnitoPlume.
Q8	I found AnitoPlume very cumbersome (slow or complicated and therefore inefficient) to use.
Q9	I felt very confident while using AnitoPlume.
Q10	I needed to figure out a lot of things (such as hotkeys, navigating the UI) before I could confidently use AnitoPlume.

4.4 Research Workflow

This study follows a three-phase research workflow, outlined in Table 6, that guides the porting and extension of AnitoPlume from its legacy rendering implementation to the proposed rendering architecture, from initial legacy analysis through final evaluation of performance and usability.

Phase	Research Activity	Purpose / Output
1	**Legacy Pipeline Profiling and Bottleneck Analysis**	Evaluate the existing simulation and rendering architecture to establish baseline performance metrics. Profile CPU-GPU synchronization, draw call overhead, and frame pacing to identify compute and graphics pipeline bottlenecks that impede scalability.
2	**Architectural Refactoring and Pipeline Optimization**	Design and iteratively implement optimizations within the execution model and rendering pipeline. This involves exploring modern graphics paradigms (e.g., explicit command submission, reducing driver overhead) and evaluating concurrent execution strategies to maximize render throughput, adapting the architecture based on empirical performance gains.
3	**Performance Benchmarking and Interactive Evaluation**	Conduct a comparative analysis between the legacy and refactored pipelines using low-level GPU profiling metrics (e.g., shader execution time, memory bandwidth utilization). Perform user testing to correlate these hardware-level optimizations with perceived interactive responsiveness and visual consistency.

Table 6: Overview of the Research Phases, Activities, and Objectives

References

Amador Herrera, J. A., Klein, J., Liu, D., Pałubicki, W., Pirk, S., & Michels, D. L. (2024). Cyclogenesis: Simulating hurricanes and tornadoes. *ACM Transactions on Graphics (TOG)*, *43*(4), 1–16.

Asgary, A., Bonadonna, C., & Frischknecht, C. (2019). Simulation and visualization of volcanic phenomena using microsoft hololens: case of vulcano island (italy). *IEEE transactions on engineering management*, *67*(3), 545–553.

Asplund, J. (2020). *Directx 12: Performance comparison between single- and multithreaded rendering when culling multiple lights* (Unpublished master's thesis). Jönköping University, School of Engineering.

Balangue-Tarriela, M., Lagmay, A., Sarmiento, D., Vasquez, J., Baldago, M., Ybañez, R., ... others (2022). Analysis of the 2020 taal volcano tephra fall deposits from crowdsourced information and field data. *Bulletin of volcanology*, *84*(3), 35.

Biass, S., Bonadonna, C., Connor, L., & Connor, C. (2016). Tephraprob: a matlab package for probabilistic hazard assessments of tephra fallout. *Journal of Applied Volcanology*, *5*(1), 10.

Blender Foundation. (2025). *Blender - the free and open source 3D creation software.* `https://www.blender.org/`. (Accessed: 2026-07-10)

Bonadonna, C. (2006). Probabilistic modelling of tephra dispersion.

Bossard, A. (2019). High-performance graphics with directx in racket. *Information Engineering Express*, *5*(2), 83–98. doi: 10.52731/iee.v5.i2.368

Brackbill, J. U., & Ruppel, H. M. (1986). FLIP: A method for adaptively zoned, particle-in-cell calculations of fluid flows in two dimensions. *Journal of Computational Physics*, *65*(2), 314–343. Retrieved from `https://doi.org/10.1016/0021-9991(86)90211-1` doi: 10.1016/0021-9991(86)90211-1

Braun, B., Bender, J., & Thuerey, N. (2025). Adaptive phase-field-flip for very large scale two-phase fluid simulation. *ACM Transactions on Graphics (TOG)*, *44*(4), 1–23.

Capodieci, N., Cavicchioli, R., & Marongiu, A. (2022). A taxonomy of modern gpgpu programming methods: On the benefits of a unified specification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and*

Systems, *41*, 1649–1662. doi: 10.1109/tcad.2021.3082863

Cheung, D. (2024). *Applications and limitations of vulkan-layer-based instrumentation* (Unpublished master's thesis). University of Alberta (Department of Computing Science).

Connor, L. J., & Connor, C. B. (2006). Inversion is the key to dispersion: understanding eruption dynamics by inverting tephra fallout.

Daguno-Bersamina, K. (2023). *Volcanic smog detected in Taal, health advisory issued.* Retrieved from `https://www.philstar.com/headlines/2023/09/22/2298189/volcanic-smog-detected-taal-health-advisory-issued` (Online News Article. Published September 22, 2023)

Delage, E., van Wyk de Vries, B., Philippe, M., Conway, S., Morino, C., Manrique Llerena, N., ... Kristinn Helgason, J. (2021). Visualising and experiencing geological flows in virtual reality. In *Egu general assembly conference abstracts* (pp. EGU21–8801).

Del Castillo, M., Paraiso, P., Vicente, M., Jamero, M., & Narisma, G. (2020). Impacts of taal volcano phreatic eruption (12 january 2020) on the environment and population: satellite-based observations compared with historical records. *Manila Observatory*.

Del Gallego, N. P., Torre, O. D., Arquillo, C. M., Cordero, V. V., Sales, J. L., & Yarte, S. L. (2026). Interactive 3d simulation of taal volcano eruption plumes. *Applied Computing and Geosciences*, 100331.

Driedger, C. L., Ramsey, D. W., Scott, W. E., Faust, L. M., Bard, J. A., & Wold, P. (2024). Following the tug of the audience from complex to simplified hazards maps at cascade range volcanoes. *Journal of Applied Volcanology*, *13*(1), 4.

Esposti Ongaro, T., Cerminara, M., Charbonnier, S. J., Lube, G., & Valentine, G. A. (2020). A framework for validation and benchmarking of pyroclastic current models. *Bulletin of Volcanology*, *82*, 59. doi: 10.1007/s00445-020-01386-4

Fearnley, C., & Beaven, S. (2018). Volcano alert level systems: managing the challenges of effective volcanic crisis communication. *Bulletin of Volcanology*, *80*(5), 46.

Fedkiw, R. (2003). Simulating natural phenomena. In *Geometric level set methods in imaging, vision, and graphics* (pp. 461–479). Springer.

Flores, H. (2019). *Taal volcano showing signs of activity?* Retrieved from `https://www.philstar.com/nation/2019/12/02/1973465/taal-volcano-showing-signs-activity` (Online News Article.)

Folch, A., Costa, A., & Macedonio, G. (2009). Fall3d: A computational model for transport and deposition of volcanic ash. *Computers & Geosciences*, *35*(6), 1334–1342.

Folch, A., Costa, A., & Macedonio, G. (2016). Fplume-1.0: An integral volcanic plume model accounting for ash aggregation. *Geoscientific Model Devel-*

opment, *9*(1), 431–450. Retrieved from https://gmd.copernicus.org/articles/9/431/2016/ doi: 10.5194/gmd-9-431-2016

Folch, A., Mingari, L., Gutierrez, N., Hanzich, M., Macedonio, G., & Costa, A. (2020). Fall3d-8.0: a computational model for atmospheric transport and deposition of particles, aerosols and radionuclides–part 1: Model physics and numerics. *Geoscientific Model Development*, *13*, 1431–1458. doi: 10.5194/gmd-13-1431-2020

George, D. L., & Iverson, R. M. (2014, 10). A depth-averaged debris-flow model that includes the effects of evolving dilatancy. II. numerical predictions and experimental tests. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, *470*(2170), 20130820. Retrieved from https://doi.org/10.1098/rspa.2013.0820 doi: 10.1098/rspa.2013.0820

Guerrero Tello, J. F., Coltelli, M., Marsella, M., Celauro, A., & Palenzuela Baena, J. A. (2022). Convolutional neural network algorithms for semantic segmentation of volcanic ash plumes using visible camera imagery. *Remote Sensing*, *14*(18), 4477. doi: 10.3390/rs14184477

Hansen, K., Barker, S., Illsley-Kemp, F., Anslow, C., Mueller, C., & Leonard, G. (2025). Volcanovr: A virtual reality environment for volcanic data visualisation and communication. *Volcanica*, *8*(1), 175–187.

Ioannidis, C., & Boutsi, A.-M. (2020). Multithreaded rendering for cross-platform 3D visualization based on Vulkan API. In *International archives of the photogrammetry, remote sensing and spatial information sciences* (Vol. XLIV-4/W1-2020, pp. 57–62). Göttingen: Copernicus GmbH.

Iverson, R. M., & George, D. L. (2014). A depth-averaged debris-flow model that includes the effects of evolving dilatancy. i. physical basis. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, *470*(2170), 20130819. doi: 10.1098/rspa.2013.0819

Jänicke, H., & Scheuermann, G. (2010). Measuring complexity in lagrangian and eulerian flow descriptions. In *Computer graphics forum* (Vol. 29, pp. 1783–1794).

Kavanagh, J. L., Engwell, S. L., & Martin, S. A. (2018). A review of laboratory and numerical modelling in volcanology. *Solid Earth*, *9*(2), 531–571.

Kerlow, I., Pedreros, G., & Albert, H. (2020). Earth girl volcano: characterizing and conveying volcanic hazard complexity in an interactive casual game of disaster preparedness and response. *Geoscience Communication*, *3*(2), 343–364.

Khronos Group. (2024). Vulkan specification (Version 1.4 ed.) [Computer software manual]. Retrieved from https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html

Kinkor, J. (2022). Easy vulkan. In *Proceedings of the 27th student eeict 2022.* Brno University of Technology, Faculty of Information Technology.

Lastic, M., Rohmer, D., Cordonnier, G., Jaupart, C., Neyret, F., & Cani, M.-P. (2022). Interactive simulation of plume and pyroclastic volcanic ejections. *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, *5*(1), 1–15.

Li, W. (2023). Synthesizing procedural landscape for volcanic eruption simulation in virtual reality. In *2023 7th international symposium on multidisciplinary studies and innovative technologies (ismsit)* (pp. 1–5).

Mallari Jr., D. (2026). *Taal volcano activity increases with 120 quakes, 113 tremors.* Retrieved from `https://newsinfo.inquirer.net/2258129/taal-volcano-activity-increases-with-120-quakes-113-tremors` (Online News Article. Published July 6, 2026)

Moyroud, N., & Portet, F. (2018). Introduction to qgis. *QGIS and generic tools*, *1*, 1–17.

Nabizadeh, M. S., Roy-Chowdhury, R., Yin, H., Ramamoorthi, R., & Chern, A. (2024). Fluid implicit particles on coadjoint orbits. *arXiv preprint arXiv:2406.01936*.

Nave, R., Isaia, R., Vilardo, G., & Barclay, J. (2010). Re-assessing volcanic hazard maps for improving volcanic risk communication: application to stromboli island, italy. *Journal of Maps*, *6*(1), 260–269.

Nishita, T., & Dobashi, Y. (2001). Modeling and rendering of various natural phenomena consisting of particles. In *Proceedings. computer graphics international 2001* (pp. 149–156).

Nowrouzezahrai, D. (2006). *Vortex based smoke simulation and control.*

The OpenGL graphics system: A specification (Version 4.6 (Core Profile) ed.) [Computer software manual]. (2022, May). Retrieved from `https://registry.khronos.org/OpenGL/specs/gl/glspec46.core.pdf`

openTopography. (2025). *OpenTopography.* `https://opentopography.org/`. (Accessed: 2026-07-10)

Pardini, F., Barsotti, S., Bonadonna, C., de' Michieli Vitturi, M., Folch, A., Mastin, L., ... Prata, A. T. (2024). Dynamics, monitoring, and forecasting of tephra in the atmosphere. *Reviews of Geophysics*, *62*(4), e2023RG000808. Retrieved from `https://doi.org/10.1029/2023RG000808` doi: 10.1029/2023RG000808

Pfaff, T., Thuerey, N., & Gross, M. (2012). Lagrangian vortex sheets for animating fluids. *ACM Transactions on Graphics (TOG)*, *31*(4). doi: 10.1145/2185520.2185608

PHIVOLCS. (2024). *Vmepd: Eruption history.* Retrieved from `https://wovodat.phivolcs.dost.gov.ph/volcan/erupt-history?volcan=580&sdate=&edate=&btn-search=`

Ping, L., Qi, S., Chen, W., Jie, G., Yanwen, G., & Wenzhe, S. (2026, March). Modern graphics APIs: Design principles, a use case, and new perspectives. *ZTE Communications*, *24*(1), 97–106. Re-

trieved from https://www.zte.com.cn/content/dam/zte-site/res-www-zte-com-cn/mediares/magazine/publication/com_en/article/en202601/20260113.pdf

Poland, M. P., & Anderson, K. R. (2020). Partly cloudy with a chance of lava flows: Forecasting volcanic eruptions in the twenty-first century. *Journal of Geophysical Research: Solid Earth*, *125*(1), e2018JB016974.

Preppernau, C. A., & Jenny, B. (2015). Three-dimensional versus conventional volcanic hazard maps. *Natural Hazards*, *78*(2), 1329–1347.

Pretorius, P. C., Gain, J., Lastic, M., Cordonnier, G., Chen, J., Rohmer, D., & Cani, M.-P. (2024). Volcanic skies: coupling explosive eruptions with atmospheric simulation to create consistent skyscapes. In *Computer graphics forum* (Vol. 43, p. e15034).

Reeves, W. T. (1998). Particle systems—a technique for modeling a class of fuzzy objects. In *Seminal graphics: pioneering efforts that shaped the field* (pp. 203–220).

Renzo, A. (2020). *List: Jan. 14 class suspensions due to taal eruption.* Online News Article. Retrieved from https://newsinfo.inquirer.net/1212526/list-january-14-class-suspensions-due-to-taal-eruption

Rossant, C., & Rougier, N. P. (2021). High-performance interactive scientific visualization with datoviz via the vulkan low-level gpu api. *Authorea Preprints*. doi: 10.22541/au.162129047.76547134/v1

Roth, M., & Guritz, R. (1995). Visualization of volcanic ash clouds. *IEEE computer graphics and applications*, *15*(4), 34–39.

Saidi, M. S., Rismanian, M., Monjezi, M., Zendehbad, M., & Fatehiboroujeni, S. (2014). Comparison between lagrangian and eulerian approaches in predicting motion of micron-sized particles in laminar flows. *Atmospheric Environment*, *89*, 199–206.

Sanchez-Gonzalez, A., Godwin, J., Pfaff, T., Ying, R., Leskovec, J., & Battaglia, P. (2020). Learning to simulate complex physics with graph networks. In *International conference on machine learning* (pp. 8459–8468).

Selle, A., Rasmussen, N., & Fedkiw, R. (2005). A vortex particle method for smoke, water and explosions. In *Acm siggraph 2005 papers* (pp. 910–914).

Shiraef, J. (2016). *An exploratory study of high performance graphics application programming interfaces* (Unpublished master's thesis). University of Tennessee at Chattanooga. (UTC Digital Commons)

Shiraef, J. A. (2016). *An exploratory study of high performance graphics application programming interfaces* (Master's thesis, University of Tennessee at Chattanooga). Retrieved from https://scholar.utc.edu/theses/446/

Smithsonian Institution. (2026). *Taal.* Retrieved from https://volcano.si.edu/volcano.cfm?vn=273070 (Online Website page)

Stam, J. (2023). Stable fluids. In *Seminal graphics papers: Pushing the boundaries, volume 2* (pp. 779–786).

Stein, A., Draxler, R., Rolph, G., Stunder, B., Cohen, M., Ngan, F., ... others (2015). *Noaa's hysplit atmospheric transport and dispersion modeling system, b. am. meteorol. soc., 96, 2059–2077.*

Stora, D., Agliati, P.-O., Cani, M.-P., Neyret, F., & Gascuel, J.-D. (1999). Animating lava flows. In *Graphics interface (gi'99) proceedings* (pp. 203–210).

Swetha, G., Datla, R., & Vishnu, C. (2023, July). MS-VACSNet: A network for multi-scale volcanic ash cloud segmentation in remote sensing images. In *2023 18th international conference on machine vision and applications (mva)* (pp. 1–6). IEEE.

TechArx Gaming Staff. (2016). *Understanding graphics apis.* Retrieved from `https://techarx.com/understanding-graphics-apis/` (Online Article Image.)

Thompson, M. A., Lindsay, J. M., & Gaillard, J.-C. (2015). The influence of probabilistic volcanic hazard map properties on hazard communication. *Journal of Applied Volcanology*, *4*(1), 6.

Thompson, M. A., Lindsay, J. M., & Leonard, G. S. (2017). More than meets the eye: volcanic hazard map design and visual communication. In *Observing the volcano world: volcano crisis communication* (pp. 621–640). Springer.

Tibaldi, A., Bonali, F. L., Vitello, F., Delage, E., Nomikou, P., Antoniou, V., ... Whitworth, M. (2020). Real world–based immersive virtual reality for research, teaching and communication in volcanology. *Bulletin of Volcanology*, *82*(5), 38.

Torrisi, F., Corradino, C., Cariello, S., & Del Negro, C. (2024). Enhancing detection of volcanic ash clouds from space with convolutional neural networks. *Journal of Volcanology and Geothermal Research*, *448*, 108046. doi: 10.1016/j.jvolgeores.2024.108046

Trujillo-Vela, M. G., Ramos-Cañon, A. M., Escobar-Vargas, J. A., & Galindo-Torres, S. A. (2022). An overview of debris-flow mathematical modelling. *Earth-Science Reviews*, *232*, 104135. Retrieved from `https://doi.org/10.1016/j.earscirev.2022.104135` doi: 10.1016/j.earscirev.2022.104135

Unterguggenberger, J., Kerbl, B., & Wimmer, M. (2022). The road to vulkan: Teaching modern low-level apis in introductory graphics courses. In *Eurographics 2022 - education papers.* The Eurographics Association. doi: 10.2312/eged.20221043

U.S. Geological Survey. (2018). *Standard forest metrics - USGS 3DEP LiDAR.* Retrieved from `https://eros.usgs.gov/doi-remote-sensing-activities/2018/usgs/standard-forest-metrics-usgs-3dep-lidar` (Online Article/Report.)

Vimont, U., Gain, J., Lastic, M., Cordonnier, G., Abiodun, B., & Cani, M.-P. (2020). Interactive meso-scale simulation of skyscapes. In *Computer graphics forum* (Vol. 39, pp. 585–596).

Wilkes, T. C., Pering, T. D., & McGonigle, A. J. (2022). Semantic segmentation of

explosive volcanic plumes through deep learning. *Computers & Geosciences*, *168*, 105216. doi: 10.1016/j.cageo.2022.105216

Zhang, S., Kong, F., Li, C., Wang, C., & Qin, H. (2017). Hybrid modeling of multiphysical processes for particle-based volcano animation. *Computer Animation and Virtual Worlds*, *28*(3-4), e1758.

Zoraja, I., Bonkovic, M., Papic, V., & Sunderam, V. (2023). Vanityx: An agile 3d rendering platform supporting mixed reality. *Applied Sciences*, *13*, 5468. Retrieved from `https://doi.org/10.3390/app13095468` doi: 10.3390/app13095468